FRONTEND ENGINEERING SERIES

Building the Frontend for Your ML Backend

A Professional React, TypeScript and Zustand Application Connected to the Salary Prediction API

A STEP-BY-STEP LEARNING GUIDE

PROJECT	Salary Estimator web application
STACK	React 19 · TypeScript · Vite · Zustand · Vitest
BACKEND	FastAPI salary-prediction API (/health, /predict)
ARCHITECTURE	Layered, feature-based folder structure
SCOPE	17 steps; code type-checked, built and tested against the API

Version 1.0 · November 2026

CONTENTS

Table of Contents

Roadmap and Objectives . . . 4

Roadmap at a Glance . . . 4

Deliverables . . . 4

Prerequisites . . . 5

How the Frontend Fits the System . . . 6

The API Contract . . . 6

What Happens When the User Clicks “Predict salary” . . . 6

Why the Browser Needs CORS . . . 7

A Learning Map of Front-End Concepts . . . 8

The Layering Rule . . . 8

Part A · Foundations . . . 10

Step 1 · Scaffold the Project . . . 10

Step 2 · Design the Folder Structure . . . 10

Step 3 · Model the Data . . . 11

Step 4 · Build the API Layer . . . 13

Step 5 · Connect to the Backend . . . 15

Part B · State Management with Zustand . . . 16

Step 6 · Understand Zustand . . . 16

Step 7 · Build the Prediction Store . . . 17

Step 8 · Decide What Belongs in the Store . . . 19

Part C · Building the Interface . . . 20

Step 9 · Design Tokens and Global CSS . . . 20

Step 10 · Create Reusable UI Components . . . 21

Step 11 · The Prediction Form . . . 23

Step 12 · Results and History . . . 25

Step 13 · Compose the Application . . . 27

Part D · Quality and Delivery . . . 29

Step 14 · States, UX and Accessibility . . . 29

Step 15 · Test the Frontend . . . 29

Step 16 · Build and Deploy . . . 31

Step 17 · Next Steps and a Learning Plan . . . 32

Pitfalls and Final Checklist 32

Final Checklist 33

Appendix A · Learning Self-Assessment 34

Appendix B · Complete Stylesheets 34

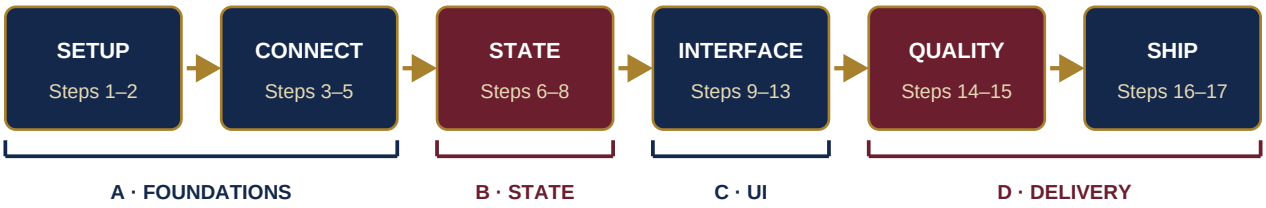
Appendix C · References 39

CHAPTER 1

Roadmap and Objectives

In the previous guide you trained a salary model and exposed it as a FastAPI service. A model that only answers `curl` commands is not yet a product. This guide shows how to build the **web interface** that lets a person fill in a job profile, send it to your API and read the estimate, using the tools that professional front-end teams use today: **React** for the interface, **TypeScript** for safety, **Vite** for the build tool and **Zustand** for state management.

The guide has a second goal: it is deliberately organised as a **learning project**. Every file has a single responsibility, every concept is introduced exactly where you need it, and each chapter ends with a place to practise. All code was type-checked with TypeScript in strict mode, built with Vite, covered by automated tests, and exercised against the real FastAPI service from the previous guide.



Roadmap at a Glance

Step	Action	Main output
1	Scaffold the project with Vite, React and TypeScript	Running dev server
2	Design the folder structure and the dependency rules	`src/` layers
3	Model the data: TypeScript types and option constants	`types/`, `constants/`
4	Build the API layer: fetch client and error handling	`api/client.ts`
5	Connect to the backend: CORS, environment variables, smoke test	Browser ↔ API works
6	Understand Zustand: stores, selectors, actions	Mental model
7	Build the prediction store with persist and devtools	`usePredictionStore`
8	Decide what belongs in global state and what does not	State taxonomy
9	Design tokens, dark mode and global CSS	`styles/`
10	Create reusable UI components	`components/ui/`
11	Build the prediction form with validation	`PredictionForm`
12	Show results and history	`ResultCard`, `HistoryList`
13	Compose the application shell	`App`, `Header`, `useHealth`
14	Loading, error and empty states; accessibility	UX checklist
15	Test the store and utilities with Vitest	Green test run
16	Build, containerise and deploy; CI pipeline	Public website
17	Next steps and a four-week learning plan	Practice roadmap

Deliverables

- A working single-page application connected to your `/predict` and `/health` endpoints.
- A layered, feature-based code base of about 25 small files that you can explain line by line.

- A Zustand store that handles form state, validation, requests, errors, cancellation and persisted history.
- Automated tests, a production build, and a deployment recipe.
- A personal learning plan with exercises of increasing difficulty.

Prerequisites

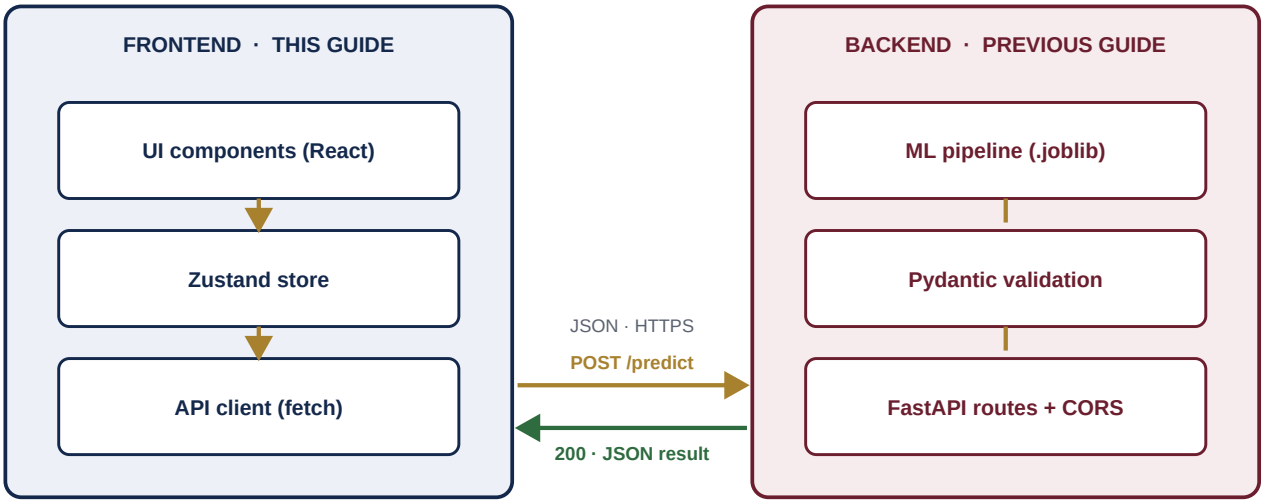
- Node.js 20 or newer (the guide was tested with Node 22) and a code editor such as VS Code.
- Your FastAPI service from the previous guide, running locally on port 8000.
- Basic JavaScript: variables, functions, arrays, objects, `async`/`await`. TypeScript and React are taught here from the ground up.

> **NOTE · VERSIONS USED**
> The code was tested with React 19.3, Vite 8.3, TypeScript 7.0, Zustand 5.0 and Vitest 5.0. The Vite template changes from release to release, so small differences in generated files are normal; the concepts and the code in this guide do not depend on them.

CHAPTER 2

How the Frontend Fits the System

Before writing code, understand the whole picture. The browser never touches the machine-learning model. It sends a small JSON message to the API and displays the JSON that comes back. Everything else in this guide is about doing that reliably, clearly and in a maintainable way.



The API Contract

The frontend and the backend are two separate programs that agree on a **contract**: which URLs exist, what must be sent and what comes back. Your TypeScript types in Step 3 are a direct copy of this contract, so a mismatch is caught by the compiler instead of by a user.

Endpoint	Method	Request body	Success response	Errors
`/health`	GET	none	`{status, model_version}`	network error if the API is down
`/predict`	POST	JSON with 8 fields (see Step 3)	`{predicted_salary_usd, typical_error_usd, model_version}`	422 with a `detail` list when a field is invalid

What Happens When the User Clicks “Predict salary”

1. The form’s `onSubmit` handler runs and calls `event.preventDefault()` so the page does not reload.
2. It calls `submit()` on the Zustand store, which first runs `validate()` on the current form values.
3. If valid, the store sets `status = "loading"`; the button shows a spinner and the result card shows a skeleton.
4. The store calls `predictSalary()` in the API layer, which uses `fetch` to POST the JSON to FastAPI.
5. FastAPI validates the body with Pydantic, runs the model and returns JSON (or a 422 error).
6. The store saves the result and a history entry, and sets `status = "success"`.
7. Every component that selected those values re-renders automatically; nothing else does.

> **KEY · THE SINGLE MOST IMPORTANT IDEA IN FRONT-END DEVELOPMENT**
>
> **UI = f(state).** You do not tell the screen what to change; you change the state, and React redraws the parts of the interface that depend on it. Every design decision in this guide supports that idea: state lives in one clearly defined place, and components are simple functions that describe how that state looks.

Why the Browser Needs CORS

Browsers apply the **same-origin policy**: a page loaded from `http://localhost:5173` may not read responses from `http://localhost:8000` unless that server explicitly allows it. The server grants permission with **CORS** headers (Cross-Origin Resource Sharing). Without them, your request reaches the API but the browser hides the answer and prints a red CORS error in the console. Step 5 adds the required lines to the backend.

CHAPTER 3

A Learning Map of Front-End Concepts

Use this table as your syllabus. Each concept is introduced at a specific place in the project, and each has a short exercise so that you practise it rather than only read about it. Tick the concepts off in Appendix A as you go.

Concept	Where you meet it	Practice task
Components and props	`Button`, `Card`, `Field`	Add a `variant="danger"` to `Button`.
JSX and composition (`children`)	`Card`, `Field`, `App`	Create a `Badge` component and use it in `Header`.
TypeScript types and generics	`types/salary.ts`, `Select`	Add a `CompanySize` label type and use it in `HistoryList`.
Controlled inputs	`PredictionForm`	Replace the year input with a slider and keep it in the store.
Events and `preventDefault`	Form `onSubmit`	Submit with the Enter key only when the form is valid.
Conditional rendering	`ResultCard` (4 states)	Add a fifth state: “stale result” after the form changes.
Lists and keys	`HistoryList`	Show only the 5 latest items with a “Show more” button.
Effects and cleanup	`useHealth`	Re-check the API every 30 seconds and clean up the timer.
Custom hooks	`useHealth`	Write `useOnlineStatus` using `navigator.onLine`.
Async code and cancellation	`api/client.ts`	Add one automatic retry for network errors.
Global state with Zustand	`usePredictionStore`	Create `useThemeStore` with a dark-mode toggle.
Immutability	`setField` (spread syntax)	Explain why `form.job_title = x` would not update the UI.
CSS variables, grid, dark mode	`styles/*.css`	Add a third colour theme.
Accessibility	Labels, `aria-*`, focus rings	Navigate the whole app with the keyboard only.
Testing	`*.test.ts`	Write a test for `clearHistory`.

The Layering Rule

A professional code base is easy to change because each layer knows only about the layer beneath it. Follow this rule strictly; it is the reason the project stays readable as it grows.

Layer (folder)	Responsibility	May import from	Must not import from
`components/features`	Screens and parts of screens tied to this app	store, hooks, ui, utils, constants	`api` directly
`components/ui`	Generic, reusable building blocks	types, constants (for `Option`)	store, api
`store`	Application state and the actions that change it	api, constants, types, utils	components
`hooks`	Reusable React logic	api, types	components
`api`	Talking to the backend; nothing else	types	store, components
`utils`, `constants`, `types`	Pure helpers and shared definitions	types only	everything else

CHAPTER 4 · STEPS 1–5

Part A · Foundations

Step 1 · Scaffold the Project

Vite is a build tool: it serves your code instantly during development and bundles it for production. The official template creates a React + TypeScript project with sensible defaults.

LISTING 1 · Create the project (terminal)

```
npm create vite@latest salary-web -- --template react-ts
cd salary-web
npm install
npm install zustand            # state management
npm install -D vitest jsdom    # test runner and a simulated browser
npm run dev                    # http://localhost:5173
```

Delete the demo files that the template created (`src/App.css`, `src/index.css`, `src/assets/`) and the sample content of `App.tsx`; you will write your own. Then replace `vite.config.ts` so that the test runner uses a simulated browser:

LISTING 2 · vite.config.ts

```
/// <reference types="vitest/config" />
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: { port: 5173 },
  test: { environment: "jsdom", globals: false },
});
```

> **NOTE · STRICT TYPESCRIPT IS YOUR FRIEND**
>
> The template enables `strict` mode and `verbatimModuleSyntax`. Strict mode makes the compiler reject unsafe code such as using a value that might be `undefined`. `verbatimModuleSyntax` requires you to write `import type { … }` when importing something that exists only as a type; you will see this pattern in every file.

Step 2 · Design the Folder Structure

Create the folders below. The structure is organised by **responsibility**: what a file does decides where it lives. This is how real teams keep large projects navigable.

LISTING 3 · Project structure

```
salary-web/
├── index.html
├── package.json
├── vite.config.ts
├── .env.development           # VITE_API_URL=http://localhost:8000
├── .env.production            # VITE_API_URL=https://your-api-host
└── src/
    ├── main.tsx               # entry point: mounts <App /> into index.html
    ├── App.tsx                # page layout, no business logic
    ├── api/
    │   ├── client.ts          # fetch wrapper: base URL, timeout, errors
    │   └── salary.ts          # one function per endpoint
    ├── types/
    │   └── salary.ts          # TypeScript copy of the API contract
    ├── constants/
    │   └── options.ts         # dropdown options and default form values
```

```
    ├── store/
    │   ├── usePredictionStore.ts
    │   └── usePredictionStore.test.ts
    ├── hooks/
    │   └── useHealth.ts        # API online/offline indicator
    ├── utils/
    │   ├── format.ts           # currency and date formatting
    │   ├── validate.ts         # form validation
    │   └── format.test.ts
    ├── components/
    │   ├── ui/                 # Button, Card, Field, Select  (generic)
    │   ├── layout/             # Header
    │   └── features/           # PredictionForm, ResultCard, HistoryList
    └── styles/                 # tokens.css, base.css, layout.css, components.css
```

NAMING CONVENTIONS

- **Components** are `PascalCase` functions in files with the same name: `ResultCard.tsx` exports `ResultCard`.
- **Hooks** (including Zustand stores) start with `use`: `useHealth`, `usePredictionStore`. React relies on this prefix.
- **Everything else** is `camelCase`; types and interfaces are `PascalCase`; constants are `UPPER_SNAKE_CASE`.
- Keep files small. If a file grows beyond roughly 100 lines, ask whether it has two responsibilities.

Step 3 · Model the Data

Start with data, not screens. TypeScript **types** describe the exact shape of what travels between the browser and the API. They disappear at build time, but until then the compiler checks every use. The unions such as `"EN" | "MI" | "SE" | "EX"` mirror the `Literal[...]` fields in the backend's Pydantic schema, so an invalid experience level cannot even be written.

LISTING 4 · src/types/salary.ts

```ts
export type ExperienceLevel = "EN" | "MI" | "SE" | "EX";
export type EmploymentType = "FT" | "PT" | "CT" | "FL";
export type CompanySize = "S" | "M" | "L";
export type RemoteRatio = 0 | 50 | 100;

/** Mirrors `SalaryRequest` in app/schemas.py (backend). */
export interface SalaryRequest {
  work_year: number;
  experience_level: ExperienceLevel;
  employment_type: EmploymentType;
  job_title: string;
  employee_residence: string;
  remote_ratio: RemoteRatio;
  company_location: string;
  company_size: CompanySize;
}

/** Mirrors `SalaryResponse` in app/schemas.py (backend). */
export interface SalaryResponse {
  predicted_salary_usd: number;
  typical_error_usd: number;
  model_version: string;
}

export interface HealthResponse {
  status: string;
  model_version: string;
}

export interface HistoryEntry {
  id: string;
  createdAt: string; // ISO date string
  input: SalaryRequest;
  result: SalaryResponse;
}
```

The **constants** file holds everything the form needs to display: the allowed values, their human-friendly labels, and the default form. Keeping this data out of components means a label change is a one-line edit in one place.

LISTING 5 · src/constants/options.ts

```
import type {
  CompanySize,
  EmploymentType,
  ExperienceLevel,
  RemoteRatio,
  SalaryRequest,
} from "../types/salary";

export interface Option<T> {
  value: T;
  label: string;
}

export const EXPERIENCE_OPTIONS: Option<ExperienceLevel>[] = [
  { value: "EN", label: "Entry level" },
  { value: "MI", label: "Mid level" },
  { value: "SE", label: "Senior" },
  { value: "EX", label: "Executive" },
];

export const EMPLOYMENT_OPTIONS: Option<EmploymentType>[] = [
  { value: "FT", label: "Full-time" },
  { value: "PT", label: "Part-time" },
  { value: "CT", label: "Contract" },
  { value: "FL", label: "Freelance" },
];

export const COMPANY_SIZE_OPTIONS: Option<CompanySize>[] = [
  { value: "S", label: "Small" },
  { value: "M", label: "Medium" },
  { value: "L", label: "Large" },
];

export const REMOTE_OPTIONS: Option<RemoteRatio>[] = [
  { value: 0, label: "On-site" },
  { value: 50, label: "Hybrid" },
  { value: 100, label: "Fully remote" },
];

const JOB_TITLES = [
  "Data Engineer",
  "Data Scientist",
  "Data Analyst",
  "Machine Learning Engineer",
  "Analytics Engineer",
  "Research Scientist",
  "Data Architect",
  "Data Science Manager",
  "AI Scientist",
  "Applied Scientist",
  "Data Analytics Lead",
];
export const JOB_TITLE_OPTIONS: Option<string>[] = JOB_TITLES.map((t) => ({
  value: t,
  label: t,
}));

export const COUNTRY_OPTIONS: Option<string>[] = [
  ["US", "United States"],
  ["GB", "United Kingdom"],
  ["CA", "Canada"],
  ["IN", "India"],
  ["DE", "Germany"],
  ["FR", "France"],
  ["ES", "Spain"],
  ["NL", "Netherlands"],
  ["AU", "Australia"],
```

```
  ["BR", "Brazil"],
  ["PL", "Poland"],
  ["PT", "Portugal"],
  ["EG", "Egypt"],
  ["AE", "United Arab Emirates"],
  ["NG", "Nigeria"],
  ["SG", "Singapore"],
].map(([value, label]) => ({ value, label }));

export const DEFAULT_FORM: SalaryRequest = {
  work_year: 2023,
  experience_level: "SE",
  employment_type: "FT",
  job_title: "Data Scientist",
  employee_residence: "US",
  remote_ratio: 100,
  company_location: "US",
  company_size: "M",
};
```

TIP · KEEP THE LISTS HONEST

The job-title list contains the most common titles from your EDA. The backend accepts any title and pools unknown ones, so you can safely extend the list later. A good extension is a backend endpoint `GET /options` that returns the lists, so the frontend never gets out of date.

Step 4 · Build the API Layer

All network code lives in one folder. Components and stores never call `fetch` directly; they call small, typed functions. This gives you one place to change the base URL, add an authentication header, or switch to another HTTP library.

LISTING 6 · src/api/client.ts

```
const BASE_URL = (import.meta.env.VITE_API_URL ?? "http://localhost:8000").replace(
  /\/$/,
  "",
);
const TIMEOUT_MS = 10_000;

export class ApiError extends Error {
  status: number; // 0 = network error / timeout / cancelled

  constructor(message: string, status: number) {
    super(message);
    this.name = "ApiError";
    this.status = status;
  }
}

interface ValidationItem {
  loc: (string | number)[];
  msg: string;
}

/** Turn any error response (FastAPI 422, 500, ...) into one readable sentence. */
function toMessage(status: number, body: unknown): string {
  if (typeof body === "object" && body !== null && "detail" in body) {
    const detail = (body as { detail: unknown }).detail;
    if (Array.isArray(detail)) {
      return (detail as ValidationItem[])
        .map((d) => `${d.loc.slice(1).join(".")}: ${d.msg}`)
        .join("; ");
    }
    if (typeof detail === "string") return detail;
  }
  if (status >= 500) return "The server had a problem. Please try again later.";
  return `Request failed (${status}).`;
```

```
}

export async function request<T>(
  path: string,
  init: RequestInit = {},
  signal?: AbortSignal,
): Promise<T> {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), TIMEOUT_MS);
  signal?.addEventListener("abort", () => controller.abort(), { once: true });

  try {
    const res = await fetch(`${BASE_URL}${path}`, {
      ...init,
      headers: { "Content-Type": "application/json", ...init.headers },
      signal: controller.signal,
    });
    const body: unknown = await res.json().catch(() => null);
    if (!res.ok) throw new ApiError(toMessage(res.status, body), res.status);
    return body as T;
  } catch (err) {
    if (err instanceof ApiError) throw err;
    if (err instanceof DOMException && err.name === "AbortError") {
      throw new ApiError("The request timed out or was cancelled.", 0);
    }
    throw new ApiError("Cannot reach the server. Is the API running?", 0);
  } finally {
    clearTimeout(timer);
  }
}
```

Piece	What it does	Why it matters
`import.meta.env.VITE_API_URL`	Reads the API address from an environment file	The same code works on your laptop and in production.
`AbortController` + **timeout**	Cancels a request that takes longer than 10 seconds	The UI never waits forever.
`class ApiError`	One error type with a readable `message` and a `status`	Components show `error.message`; they never parse responses.
`toMessage()`	Converts FastAPI's 422 `detail` list into one sentence	Users see "experience_level: Input should be …".
`res.json().catch(() => null)`	Survives responses that are not JSON	A crashed server cannot crash the interface.
Generic `request`	Returns a value typed by the caller	`predictSalary` returns `SalaryResponse`, checked by the compiler.

LISTING 7 · src/api/salary.ts

```
import { request } from "./client";
import type { HealthResponse, SalaryRequest, SalaryResponse } from "../types/salary";

export function predictSalary(payload: SalaryRequest, signal?: AbortSignal) {
  return request<SalaryResponse>(
    "/predict",
    { method: "POST", body: JSON.stringify(payload) },
    signal,
  );
}

export function getHealth(signal?: AbortSignal) {
  return request<HealthResponse>("/health", {}, signal);
}
```

> **WARNING · ENVIRONMENT VARIABLES ARE PUBLIC IN THE BROWSER**
> Anything prefixed with `VITE_` is copied into the JavaScript that every visitor downloads. Put only non-secret values there, such as the API address. Never place passwords or private API keys in front-end code; secrets belong on the server.

Step 5 · Connect to the Backend

Add CORS permission to the FastAPI application from the previous guide. Only the exact origin of your frontend is allowed; the list is read from an environment variable so production can use a different value without changing code.

LISTING 8 · Backend: additions to app/main.py

```python
import os
from fastapi.middleware.cors import CORSMiddleware

ALLOWED_ORIGINS = os.getenv("ALLOWED_ORIGINS", "http://localhost:5173").split(",")

app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,      # exact origins only; never "*" in production
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)
```

Now run both programs in two terminals and check the connection before building any interface.

LISTING 9 · Run backend and frontend (two terminals)

```
# terminal 1: backend
uvicorn app.main:app --reload                # http://localhost:8000/docs

# terminal 2: frontend
npm run dev                                  # http://localhost:5173

# the browser performs this "preflight" check automatically before a POST:
curl -i -X OPTIONS http://localhost:8000/predict \
  -H "Origin: http://localhost:5173" \
  -H "Access-Control-Request-Method: POST"    # expect: access-control-allow-origin
```

Open the browser DevTools (F12), choose the **Network** tab, press the button in your finished app and click the `predict` request. Learning to read this tab (URL, method, status, request payload, response) is the most valuable debugging skill you can build.

COMMON CONNECTION PROBLEMS

Symptom	Cause	Fix
Console: “blocked by CORS policy”	Backend does not allow your origin	Add the exact origin (scheme, host, port) to `ALLOWED_ORIGINS`.
“Cannot reach the server”	API not running, wrong port or URL	Start uvicorn; check `VITE_API_URL`; restart `npm run dev` after editing `.env`.
Status 422	Data does not satisfy the schema	Read the message; compare with `types/salary.ts` and `schemas.py`.
Works locally, fails online	Page is HTTPS but API is HTTP (mixed content)	Serve the API over HTTPS too.
Status 404	Wrong path or missing prefix	Compare with `/docs` on the backend.

CHAPTER 5 · STEPS 6–8

Part B · State Management with Zustand

Step 6 · Understand Zustand

State is data that changes over time and that the screen depends on: what the user typed, whether a request is running, the last result. React's built-in `useState` keeps state inside one component. When several distant components need the same data, you need a shared place for it. **Zustand** (German for "state") is a small library that provides exactly that: a **store** that lives outside React, which components *subscribe* to through a hook.

LISTING 10 · The whole idea in ten lines (not part of the project)

```
import { create } from "zustand";

interface CounterState {
  count: number;
  increment: () => void;
}

export const useCounter = create<CounterState>()((set) => ({
  count: 0,
  increment: () => set((s) => ({ count: s.count + 1 })),
}));

// inside any component, no Provider needed:
//   const count = useCounter((s) => s.count);          // subscribe to one value
//   const increment = useCounter((s) => s.increment);  // actions are stable
```

Concept	Meaning
`create`	Builds the store and returns a hook (`useCounter`). The state and the actions that change it live together.
`set`	Updates the state. Pass an object to merge, or a function `(s) => ({…})` when the new value depends on the old one. Always return **new** objects; never mutate.
`get`	Reads the current state inside an action (used in `submit` to read the form).
Selector	The function you pass to the hook. The component re-renders only if the selected value changes.
`useShallow`	Lets you select several values as one object without causing extra re-renders.
Middleware	Wrappers that add features: `devtools` (inspect state in Redux DevTools) and `persist` (save to `localStorage`).

WHY ZUSTAND FOR THIS PROJECT?

Tool	Best for	Trade-off
`useState`	State used by one component or a parent and its children	Passing it through many levels ("prop drilling") becomes painful.
React Context	Rarely changing values: theme, current user	Every consumer re-renders on any change; no built-in devtools or persistence.
Redux Toolkit	Very large teams and complex workflows	More concepts and boilerplate to learn first.
Zustand	**Shared client state with minimal code**	Fewer enforced conventions; discipline is up to you.
TanStack Query	Caching and refetching of server data	A second library; ideal next step (see Step 8).

Step 7 · Build the Prediction Store

The store is the brain of the application. It holds the form values, validation errors, the request **status**, the result, the error message and the history, plus every action that changes them. Read the code once, then study the table below it.

LISTING 11 · src/store/usePredictionStore.ts

```ts
import { create } from "zustand";
import { devtools, persist } from "zustand/middleware";
import { ApiError } from "../api/client";
import { predictSalary } from "../api/salary";
import { DEFAULT_FORM } from "../constants/options";
import type { HistoryEntry, SalaryRequest, SalaryResponse } from "../types/salary";
import { validate, type FormErrors } from "../utils/validate";

type Status = "idle" | "loading" | "success" | "error";

const MAX_HISTORY = 10;
let activeRequest: AbortController | null = null; // lives outside React on purpose

interface PredictionState {
  // data
  form: SalaryRequest;
  errors: FormErrors;
  status: Status;
  result: SalaryResponse | null;
  error: string | null;
  history: HistoryEntry[];
  // actions
  setField: <K extends keyof SalaryRequest>(key: K, value: SalaryRequest[K]) => void;
  resetForm: () => void;
  submit: () => Promise<void>;
  loadFromHistory: (id: string) => void;
  clearHistory: () => void;
}

export const usePredictionStore = create<PredictionState>()(
  devtools(
    persist(
      (set, get) => ({
        form: DEFAULT_FORM,
        errors: {},
        status: "idle",
        result: null,
        error: null,
        history: [],

        setField: (key, value) =>
          set(
            (s) => ({
              form: { ...s.form, [key]: value },
              errors: { ...s.errors, [key]: undefined },
            }),
            false,
            "form/setField",
          ),

        resetForm: () =>
          set(
            { form: DEFAULT_FORM, errors: {}, status: "idle", result: null, error: null },
            false,
            "form/reset",
          ),

        submit: async () => {
          const form = get().form;
          const errors = validate(form);
          if (Object.keys(errors).length > 0) {
            set({ errors }, false, "submit/invalid");
            return;
```

```
          }

          activeRequest?.abort(); // cancel the previous request, if any
          const mine = new AbortController();
          activeRequest = mine;
          set({ status: "loading", error: null }, false, "submit/start");

          try {
            const result = await predictSalary(form, mine.signal);
            if (activeRequest !== mine) return; // a newer request replaced this one
            const entry: HistoryEntry = {
              id: crypto.randomUUID(),
              createdAt: new Date().toISOString(),
              input: form,
              result,
            };
            set(
              (s) => ({
                status: "success",
                result,
                history: [entry, ...s.history].slice(0, MAX_HISTORY),
              }),
              false,
              "submit/success",
            );
          } catch (err) {
            if (activeRequest !== mine) return;
            const message =
              err instanceof ApiError ? err.message : "Something went wrong.";
            set({ status: "error", error: message }, false, "submit/error");
          }
        },

        loadFromHistory: (id) => {
          const entry = get().history.find((h) => h.id === id);
          if (entry)
            set(
              {
                form: entry.input,
                result: entry.result,
                status: "success",
                error: null,
                errors: {},
              },
              false,
              "history/load",
            );
        },

        clearHistory: () => set({ history: [] }, false, "history/clear"),
      }),
      {
        name: "salary-estimator:v1",
        version: 1,
        // only these two fields survive a page refresh
        partialize: (s) => ({ form: s.form, history: s.history }),
      },
    ),
    { name: "prediction-store", enabled: import.meta.env.DEV },
  ),
);
```

Part of the store	What it does	Concept
`type Status = "idle" | "loading" | …`	One variable describes the request phase instead of several booleans that can contradict each other	State machine
`create()(…)`	Types the whole store; the empty `()` is required for the TypeScript pattern with middleware	Generics
`setField`	Copies the form with the spread operator and replaces one key; also clears that field's error	Immutability
`submit`	Validates, cancels any previous request, sets `loading`, awaits the API, then stores success or error	Async action
`activeRequest`	A module variable holding the latest `AbortController`; a late response from an older request is ignored	Race condition
`persist`	Saves `form` and `history` to `localStorage`; `partialize` chooses what to save, `name` and `version` allow safe upgrades	Persistence
`devtools`	Every `set` has a label such as `submit/success`, visible in Redux DevTools; enabled only in development	Debugging
`loadFromHistory`	Copies an old entry's input and result back into the active state	Derived workflow

WARNING · THE RACE CONDITION YOU JUST AVOIDED

Imagine the user clicks twice quickly. Request A is slow, request B is fast. Without protection, A's answer arrives last and overwrites B's, so the screen shows the result of the *older* click. Aborting the previous request and checking `activeRequest !== mine` guarantees that only the latest request may update the UI.

Step 8 · Decide What Belongs in the Store

A common beginner mistake is to put every variable into the global store. Choose the simplest home for each piece of state:

Kind of state	Example in this project	Where it should live
Server state (copy of remote data)	The prediction result	Here: Zustand for simplicity. In larger apps: TanStack Query cache.
Shared client state	Form values, history	Zustand store
Local UI state	Is a dropdown open? The API health indicator	`useState` or a custom hook (`useHealth`)
URL state	Which page or tab is open	The router (React Router), so links and back button work
Derived data	Formatted salary, label of a country	Computed during render; do not store it

TIP · RULE OF THUMB

Start with local `useState`. Move state into the store only when two components that are not parent and child need it, or when it must survive navigation or a page refresh.

CHAPTER 6 · STEPS 9–13

Part C · Building the Interface

Step 9 · Design Tokens and Global CSS

A professional interface is built from a small set of **design tokens**: named values for colours, radii and shadows stored as CSS variables. Components never use raw colours; they use `var(--primary)`. Changing the look of the entire application, or adding dark mode, then means editing one file. Create four small stylesheets so each file has one job.

File	Job
`tokens.css`	Colour, radius and shadow variables; the dark-mode overrides
`base.css`	Resets and typography for the whole page
`layout.css`	Page structure: top bar, two-column grid, responsive breakpoints
`components.css`	Look of buttons, cards, fields, results and history (Appendix B)

LISTING 12 · src/styles/tokens.css

```
:root {
  --bg: #f4f2ec;
  --surface: #ffffff;
  --text: #1b2433;
  --muted: #5d6676;
  --primary: #14284b;
  --primary-text: #ffffff;
  --accent: #a8812f;
  --border: #d9d2c0;
  --danger: #8a1f2d;
  --danger-bg: #f8e9ec;
  --ok: #2e6b3f;
  --radius: 10px;
  --shadow: 0 1px 2px rgb(20 40 75 / 8%), 0 6px 18px rgb(20 40 75 / 6%);
  color-scheme: light;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0e1626;
    --surface: #16223a;
    --text: #e8ecf5;
    --muted: #a3adc2;
    --primary: #c9a24a;
    --primary-text: #0e1626;
    --accent: #c9a24a;
    --border: #2a3855;
    --danger: #ff9aa8;
    --danger-bg: #3a1c27;
    --ok: #7cd09a;
    color-scheme: dark;
  }
}
```

- **Dark mode** costs only a `@media (prefers-color-scheme: dark)` block that redefines the same variables; no component changes.
- **Responsive layout** uses CSS Grid with a single `@media (max-width: 800px)` rule that stacks the columns on phones.
- **Accessibility in CSS:** visible `:focus-visible` rings, sufficient contrast, and `prefers-reduced-motion` to stop animations for users who ask for it.

- The complete `base.css`, `layout.css` and `components.css` are in Appendix B.

Import the four files once, in `main.tsx` (Step 13). The order matters: tokens first, because the other files use them.

Step 10 · Create Reusable UI Components

A **component** is a function that receives **props** (its inputs) and returns **JSX** (a description of the interface). Four generic components cover everything the forms and cards need. They know nothing about salaries; you could copy them into any other project.

LISTING 13 · src/components/ui/Button.tsx

```tsx
import type { ButtonHTMLAttributes } from "react";

interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: "primary" | "ghost";
  loading?: boolean;
}

export function Button({
  variant = "primary",
  loading = false,
  disabled,
  children,
  ...rest
}: ButtonProps) {
  return (
    <button
      className={`btn btn--${variant}`}
      disabled={disabled || loading}
      aria-busy={loading}
      {...rest}
    >
      {loading && <span className="spinner" aria-hidden="true" />}
      {children}
    </button>
  );
}
```

`ButtonHTMLAttributes` *gives the component every native button prop (*`type`, `onClick`, *…) for free.* `...rest` *forwards them to the real , and* `aria-busy` *tells screen readers that work is in progress.*

LISTING 14 · src/components/ui/Card.tsx

```tsx
import type { ReactNode } from "react";

interface CardProps {
  title: string;
  action?: ReactNode;
  children: ReactNode;
}

export function Card({ title, action, children }: CardProps) {
  return (
    <section className="card">
      <header className="card__header">
        <h2>{title}</h2>
        {action}
      </header>
      {children}
    </section>
  );
}
```

LISTING 15 · src/components/ui/Field.tsx

```tsx
import type { ReactNode } from "react";

interface FieldProps {
  id: string;
  label: string;
  error?: string;
  children: ReactNode;
}

export function Field({ id, label, error, children }: FieldProps) {
  return (
    <div className="field">
      <label htmlFor={id}>{label}</label>
      {children}
      {error && (
        <p className="field__error" id={`${id}-error`} role="alert">
          {error}
        </p>
      )}
    </div>
  );
}
```

`children` *is whatever you put between the opening and closing tags.* `Field` *supplies the label and error message; the caller supplies the control. This is called* ***composition*** *and is React's main tool for reuse.* `htmlFor` *links the label to the input so that clicking the label focuses the control and screen readers announce it.*

LISTING 16 · src/components/ui/Select.tsx

```tsx
import type { Option } from "../../constants/options";

interface SelectProps<T extends string | number> {
  id: string;
  value: T;
  options: Option<T>[];
  onChange: (value: T) => void;
  invalid?: boolean;
}

/** Generic: works for string and number option lists and keeps the exact type. */
export function Select<T extends string | number>({
  id,
  value,
  options,
  onChange,
  invalid,
}: SelectProps<T>) {
  return (
    <select
      id={id}
      value={value}
      aria-invalid={invalid}
      aria-describedby={invalid ? `${id}-error` : undefined}
      onChange={(e) => {
        const match = options.find((o) => String(o.value) === e.target.value);
        if (match) onChange(match.value);
      }}
    >
      {options.map((o) => (
        <option key={String(o.value)} value={o.value}>
          {o.label}
        </option>
      ))}
    </select>
  );
}
```

`Select` *is a* ***generic component****: for* `experience_level` *it infers* `T = ExperienceLevel`*, so* `onChange` *receives exactly* `"EN" | "MI" | "SE" | "EX"` *and passing the result to* `setField` *type-checks. The* `find` *step converts the browser's string*

value back to the original type, which matters for numeric options such as `0 | 50 | 100`.

Step 11 · The Prediction Form

This is the first **feature component**: it knows about salaries and connects the store to the generic UI pieces. Notice how little logic it contains; the store does the work.

LISTING 17 · src/components/features/PredictionForm.tsx

```
import { useShallow } from "zustand/react/shallow";
import {
  COMPANY_SIZE_OPTIONS,
  COUNTRY_OPTIONS,
  EMPLOYMENT_OPTIONS,
  EXPERIENCE_OPTIONS,
  JOB_TITLE_OPTIONS,
  REMOTE_OPTIONS,
} from "../../constants/options";
import { usePredictionStore } from "../../store/usePredictionStore";
import { Button } from "../ui/Button";
import { Card } from "../ui/Card";
import { Field } from "../ui/Field";
import { Select } from "../ui/Select";

export function PredictionForm() {
  // useShallow: re-render only when one of these selected values really changes
  const { form, errors, status, setField, submit, resetForm } = usePredictionStore(
    useShallow((s) => ({
      form: s.form,
      errors: s.errors,
      status: s.status,
      setField: s.setField,
      submit: s.submit,
      resetForm: s.resetForm,
    })),
  );

  return (
    <Card title="Job profile">
      <form
        noValidate
        onSubmit={(e) => {
          e.preventDefault();
          void submit();
        }}
      >
        <div className="grid">
          <Field id="experience" label="Experience level">
            <Select
              id="experience"
              value={form.experience_level}
              options={EXPERIENCE_OPTIONS}
              onChange={(v) => setField("experience_level", v)}
            />
          </Field>
          <Field id="title" label="Job title" error={errors.job_title}>
            <Select
              id="title"
              value={form.job_title}
              options={JOB_TITLE_OPTIONS}
              invalid={!!errors.job_title}
              onChange={(v) => setField("job_title", v)}
            />
          </Field>
          <Field id="employment" label="Employment type">
            <Select
              id="employment"
              value={form.employment_type}
              options={EMPLOYMENT_OPTIONS}
              onChange={(v) => setField("employment_type", v)}
            />
```

```
          </Field>
          <Field id="size" label="Company size">
            <Select
              id="size"
              value={form.company_size}
              options={COMPANY_SIZE_OPTIONS}
              onChange={(v) => setField("company_size", v)}
            />
          </Field>
          <Field id="remote" label="Work arrangement">
            <Select
              id="remote"
              value={form.remote_ratio}
              options={REMOTE_OPTIONS}
              onChange={(v) => setField("remote_ratio", v)}
            />
          </Field>
          <Field id="year" label="Work year" error={errors.work_year}>
            <input
              id="year"
              type="number"
              inputMode="numeric"
              value={form.work_year}
              aria-invalid={!!errors.work_year}
              aria-describedby={errors.work_year ? "year-error" : undefined}
              onChange={(e) => setField("work_year", Number(e.target.value))}
            />
          </Field>
          <Field
            id="residence"
            label="Employee residence"
            error={errors.employee_residence}
          >
            <Select
              id="residence"
              value={form.employee_residence}
              options={COUNTRY_OPTIONS}
              invalid={!!errors.employee_residence}
              onChange={(v) => setField("employee_residence", v)}
            />
          </Field>
          <Field id="location" label="Company location" error={errors.company_location}>
            <Select
              id="location"
              value={form.company_location}
              options={COUNTRY_OPTIONS}
              invalid={!!errors.company_location}
              onChange={(v) => setField("company_location", v)}
            />
          </Field>
        </div>

        <div className="actions">
          <Button type="submit" loading={status === "loading"}>
            Predict salary
          </Button>
          <Button type="button" variant="ghost" onClick={resetForm}>
            Reset
          </Button>
        </div>
      </form>
    </Card>
  );
}
```

- **Controlled inputs:** each control's `value` comes from the store and `onChange` writes back, so the store is always the single source of truth.
- `useShallow`**:** selecting an object creates a new object on every call; `useShallow` compares its contents, preventing needless re-renders.

- `noValidate` + **custom validation:** we show our own consistent error messages instead of the browser's pop-ups.
- `void submit()`**:** `submit` returns a Promise; `void` states that we deliberately do not wait for it inside the handler.
- **Accessibility:** `aria-invalid` and `aria-describedby` connect an invalid control to its error text, and the error has `role="alert"`.

LISTING 18 · src/utils/validate.ts

```
import type { SalaryRequest } from "../types/salary";

export type FormErrors = Partial<Record<keyof SalaryRequest, string>>;

/** Client-side checks. The backend validates again: never trust the browser alone. */
export function validate(form: SalaryRequest): FormErrors {
  const errors: FormErrors = {};
  if (!form.job_title.trim()) errors.job_title = "Choose a job title.";
  if (!/^[A-Z]{2}$/.test(form.employee_residence))
    errors.employee_residence = "Choose a country.";
  if (!/^[A-Z]{2}$/.test(form.company_location))
    errors.company_location = "Choose a country.";
  if (
    !Number.isInteger(form.work_year) ||
    form.work_year < 2020 ||
    form.work_year > 2035
  ) {
    errors.work_year = "Enter a year between 2020 and 2035.";
  }
  return errors;
}
```

*Client-side validation is for **user experience** (instant feedback); it is not security. A user can bypass the browser, which is why the backend validates every request again with Pydantic.*

Step 12 · Results and History

LISTING 19 · src/utils/format.ts

```
const usd = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 0,
});

export const formatUsd = (value: number): string => usd.format(value);

export const formatDate = (iso: string): string =>
  new Date(iso).toLocaleString("en-GB", { dateStyle: "medium", timeStyle: "short" });
```

LISTING 20 · src/components/features/ResultCard.tsx

```
import { usePredictionStore } from "../../store/usePredictionStore";
import { formatUsd } from "../../utils/format";
import { Card } from "../ui/Card";

export function ResultCard() {
  const status = usePredictionStore((s) => s.status);
  const result = usePredictionStore((s) => s.result);
  const error = usePredictionStore((s) => s.error);

  return (
    <Card title="Estimated salary">
      <div aria-live="polite">
        {status === "idle" && (
          <p className="muted">
            Fill in the profile and press <strong>Predict salary</strong>.
          </p>
```

```
        )}
        {status === "loading" && (
          <div className="skeleton" aria-label="Calculating estimate" />
        )}
        {status === "error" && (
          <p className="alert" role="alert">
            {error}
          </p>
        )}
        {status === "success" && result && (
          <>
            <p className="result__value">{formatUsd(result.predicted_salary_usd)}</p>
            <p className="muted">
              per year · typical error about ± {formatUsd(result.typical_error_usd)}
            </p>
            <p className="result__meta">Model version {result.model_version}</p>
          </>
        )}
      </div>
    </Card>
  );
}
```

`ResultCard` *shows one of four views depending on* `status`*, which is* ***conditional rendering****. It subscribes to three separate values with three selectors; changing the form never re-renders it. The wrapper with* `aria-live="polite"` *makes screen readers announce the new estimate without interrupting the user.*

LISTING 21 · src/components/features/HistoryList.tsx

```
import { useShallow } from "zustand/react/shallow";
import { COUNTRY_OPTIONS, EXPERIENCE_OPTIONS } from "../../constants/options";
import { usePredictionStore } from "../../store/usePredictionStore";
import { formatDate, formatUsd } from "../../utils/format";
import { Button } from "../ui/Button";
import { Card } from "../ui/Card";

const levelLabel = (v: string) =>
  EXPERIENCE_OPTIONS.find((o) => o.value === v)?.label ?? v;
const countryLabel = (v: string) =>
  COUNTRY_OPTIONS.find((o) => o.value === v)?.label ?? v;

export function HistoryList() {
  const { history, loadFromHistory, clearHistory } = usePredictionStore(
    useShallow((s) => ({
      history: s.history,
      loadFromHistory: s.loadFromHistory,
      clearHistory: s.clearHistory,
    })),
  );

  return (
    <Card
      title="Recent estimates"
      action={
        history.length > 0 ? (
          <Button variant="ghost" onClick={clearHistory}>
            Clear
          </Button>
        ) : null
      }
    >
      {history.length === 0 ? (
        <p className="muted">Your last 10 estimates will appear here.</p>
      ) : (
        <ul className="history">
          {history.map((h) => (
            <li key={h.id}>
              <button type="button" onClick={() => loadFromHistory(h.id)}>
                <span className="history__main">
                  {h.input.job_title} · {levelLabel(h.input.experience_level)}
                </span>
                <span className="history__sub">
```

```
                  {countryLabel(h.input.company_location)} · {formatDate(h.createdAt)}
                </span>
                <span className="history__value">
                  {formatUsd(h.result.predicted_salary_usd)}
                </span>
              </button>
            </li>
          ))}
        </ul>
      )}
    </Card>
  );
}
```

`history.map(…)` *turns an array of data into an array of elements, which is how every list in React is built. Each item needs a stable* `key` *(here the entry's* `id`*); never use the array index when items can be reordered or removed. Each row is a real , so it is focusable and works with the keyboard without extra code.*

Step 13 · Compose the Application

LISTING 22 · src/hooks/useHealth.ts

```ts
import { useEffect, useState } from "react";
import { getHealth } from "../api/salary";
import type { HealthResponse } from "../types/salary";

type HealthState =
  | { status: "checking" }
  | { status: "online"; data: HealthResponse }
  | { status: "offline" };

export function useHealth(): HealthState {
  const [state, setState] = useState<HealthState>({ status: "checking" });

  useEffect(() => {
    const controller = new AbortController();
    getHealth(controller.signal)
      .then((data) => setState({ status: "online", data }))
      .catch(() => {
        if (!controller.signal.aborted) setState({ status: "offline" });
      });
    return () => controller.abort(); // cleanup when the component unmounts
  }, []);

  return state;
}
```

A ***custom hook*** *is a function that starts with* `use` *and packages stateful logic for reuse.* `useEffect` *runs after the component appears; the function it returns is the* ***cleanup****, executed when the component disappears, which here cancels the pending request. In development React's* `StrictMode` *mounts components twice on purpose to expose missing cleanups; the* `signal.aborted` *check keeps that harmless.*

LISTING 23 · src/components/layout/Header.tsx

```ts
import { useHealth } from "../../hooks/useHealth";

export function Header() {
  const health = useHealth();
  const label =
    health.status === "checking"
      ? "Checking API…"
      : health.status === "online"
        ? `API online · model ${health.data.model_version}`
        : "API offline";

  return (
    <header className="topbar">
      <div className="topbar__inner">
        <div>
          <p className="eyebrow">Data &amp;amp; AI careers</p>
```

```
          <h1>Salary Estimator</h1>
        </div>
        <span className={`badge badge--${health.status}`} role="status">
          <span className="dot" aria-hidden="true" />
          {label}
        </span>
      </div>
    </header>
  );
}
```

LISTING 24 · src/App.tsx

```
import { HistoryList } from "./components/features/HistoryList";
import { PredictionForm } from "./components/features/PredictionForm";
import { ResultCard } from "./components/features/ResultCard";
import { Header } from "./components/layout/Header";

export default function App() {
  return (
    <>
      <Header />
      <main className="page">
        <PredictionForm />
        <div className="stack">
          <ResultCard />
          <HistoryList />
        </div>
      </main>
      <footer className="footer">
        Estimates come from a statistical model trained on public salary records. They are
        indicative only and are not an offer or a guarantee of pay.
      </footer>
    </>
  );
}
```

LISTING 25 · src/main.tsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./styles/tokens.css";
import "./styles/base.css";
import "./styles/layout.css";
import "./styles/components.css";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

`App` contains no logic at all: it only arranges components. Run `npm run dev` and you now have a complete, connected application.

CHAPTER 7 · STEPS 14–17

Part D · Quality and Delivery

Step 14 · States, UX and Accessibility

Interfaces are judged by what happens when things go wrong. Design every state deliberately.

State	What the user sees	Where it is implemented
Idle	Instruction to fill the form and press the button	`ResultCard`, `status === "idle"`
Loading	Spinner in the button, skeleton in the result; button disabled to prevent double submit	`Button loading`, `.skeleton`
Success	Large formatted salary, typical error, model version	`ResultCard`
Validation error	Message under the field, field highlighted	`Field`, `aria-invalid`
Request error	Red message with a human sentence (offline, timeout, server)	`ResultCard`, `ApiError.message`
Empty history	Friendly explanation instead of a blank box	`HistoryList`
API offline	Red badge in the header from the first second	`Header`, `useHealth`

ACCESSIBILITY CHECKLIST

Check	How to verify
Every control has a visible label linked with `htmlFor`	Click a label: its control should focus.
The whole app works with the keyboard	Use only Tab, Shift+Tab, Enter, arrow keys and Space.
Focus is always visible	Tab through the page; a ring should appear on each control.
Dynamic changes are announced	Turn on a screen reader (NVDA, VoiceOver) and submit the form.
Colour contrast is sufficient in both themes	Lighthouse audit in Chrome DevTools.
Text can be enlarged; layout does not break	Zoom the browser to 200% and resize the window.
Motion respects user settings	Enable “reduce motion” in the operating system.

Step 15 · Test the Frontend

Tests are small programs that check your code and protect you when you change it. The most valuable tests here are for the **store**, because it contains the logic. A test mocks the API module so that no network is needed, runs an action and asserts on the resulting state.

LISTING 26 · src/store/usePredictionStore.test.ts

```
import { beforeEach, describe, expect, it, vi } from "vitest";
import { ApiError } from "../api/client";
import * as api from "../api/salary";
import { usePredictionStore } from "./usePredictionStore";

vi.mock("../api/salary");

const okResponse = {
  predicted_salary_usd: 150000,
```

```
  typical_error_usd: 40000,
  model_version: "v1",
};

beforeEach(() => {
  usePredictionStore.setState(usePredictionStore.getInitialState(), true);
  vi.resetAllMocks();
});

describe("prediction store", () => {
  it("stores the result and adds a history entry on success", async () => {
    vi.mocked(api.predictSalary).mockResolvedValue(okResponse);
    await usePredictionStore.getState().submit();
    const s = usePredictionStore.getState();
    expect(s.status).toBe("success");
    expect(s.result?.predicted_salary_usd).toBe(150000);
    expect(s.history).toHaveLength(1);
  });

  it("exposes a readable message when the API fails", async () => {
    vi.mocked(api.predictSalary).mockRejectedValue(
      new ApiError("Cannot reach the server.", 0),
    );
    await usePredictionStore.getState().submit();
    const s = usePredictionStore.getState();
    expect(s.status).toBe("error");
    expect(s.error).toBe("Cannot reach the server.");
  });

  it("does not call the API when the form is invalid", async () => {
    usePredictionStore.getState().setField("work_year", 1900);
    await usePredictionStore.getState().submit();
    expect(api.predictSalary).not.toHaveBeenCalled();
    expect(usePredictionStore.getState().errors.work_year).toBeDefined();
  });
});
```

LISTING 27 · src/utils/format.test.ts

```
import { describe, expect, it } from "vitest";
import { formatUsd } from "./format";
import { validate } from "./validate";
import { DEFAULT_FORM } from "../constants/options";

describe("formatUsd", () => {
  it("formats whole dollars with a currency symbol", () => {
    expect(formatUsd(123456.7)).toBe("$123,457");
  });
});

describe("validate", () => {
  it("accepts the default form", () => {
    expect(validate(DEFAULT_FORM)).toEqual({});
  });
  it("rejects a year outside the supported range", () => {
    expect(validate({ ...DEFAULT_FORM, work_year: 1999 }).work_year).toBeDefined();
  });
});
```

LISTING 28 · Run the tests and the type-checked build

```
npm test            # vitest run: 6 tests
npm run build       # type-check + production bundle in dist/
```

> **TIP · THE TESTING LADDER**
>
> Start with pure functions (`validate`, `formatUsd`), then stores, then components with React Testing Library (render, click, assert on what the user sees), and finally end-to-end tests with Playwright that drive a real browser against a running backend.

Step 16 · Build and Deploy

`npm run build` produces the folder `dist/` with static files: HTML, one JavaScript bundle and one CSS file (about 75 kB of JavaScript after compression for this app). Any static host can serve them; there is no server code to run.

> **WARNING · BUILD-TIME VARIABLES**
>
> `VITE_API_URL` is read when you **build**, not when the page loads. Changing it requires a new build. Set it in your host's environment settings before building the production version.

Host	Good for	Notes
Vercel, Netlify, Cloudflare Pages	Fastest path; connect the Git repository	Automatic HTTPS and preview deployments for every pull request.
GitHub Pages	Free portfolio projects	Needs the correct `base` setting in `vite.config.ts` when served from a sub-path.
Nginx in Docker	Same place as your backend, full control	See the multi-stage Dockerfile below.
Object storage + CDN (S3, Azure Blob)	Cloud-native production	Configure a fallback to `index.html` for single-page apps.

LISTING 29 · Dockerfile for the frontend (multi-stage)

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

LISTING 30 · .github/workflows/ci.yml

```
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
      - run: npm test
```

PRODUCTION CHECKLIST

- Both the website and the API use **HTTPS**.
- Set `ALLOWED_ORIGINS` on the backend to the exact production origin of the website.
- Set `VITE_API_URL` to the production API address before building.
- No secrets in the bundle; review `dist/` if in doubt.
- Add a page title, `lang` attribute, favicon and a short description for search and sharing.

- Run a Lighthouse audit for performance, accessibility and best practices.

Step 17 · Next Steps and a Learning Plan

You now have a complete project. The way to turn it into skill is to change it. Follow this four-week plan, about six to eight hours per week.

Week	Focus	What to do	You can now
1	Foundations (Steps 1–5)	Type everything by hand, do not paste. Make `/health` succeed from the browser. Read the Network tab.	Call an API from a typed client.
2	State (Steps 6–8)	Build the store; write the three store tests; add `useThemeStore` with a toggle.	Design and test application state.
3	Interface (Steps 9–13)	Build all components; complete the practice tasks of Chapter 3; add one new field end to end.	Compose accessible, responsive UI.
4	Quality and shipping (Steps 14–17)	Accessibility audit, fix findings, deploy, add CI, write the README.	Ship and maintain a front end.

EXTENSION EXERCISES

Level	Exercise	New concept
Beginner	Dark-mode toggle stored in `useThemeStore` and persisted	Second store, `data-theme` attribute
Beginner	Copy-to-clipboard button on the result	Browser API, user feedback
Intermediate	Compare two profiles side by side with the difference in dollars	Derived state, layout
Intermediate	Replace form logic with React Hook Form and Zod schemas	Form libraries, schema validation
Intermediate	Replace `submit` with TanStack Query `useMutation`	Server-state caching
Intermediate	Add React Router with an “About the model” page that reads `/health`	Routing, URL state
Advanced	Add Arabic with right-to-left layout and a language switch	Internationalisation, logical CSS
Advanced	Playwright end-to-end test: fill the form, submit, assert the salary	E2E testing
Advanced	Add login with a token and an `Authorization` header in `client.ts`	Authentication, security

CHAPTER 8

Pitfalls and Final Checklist

Pitfall	Symptom	Prevention
Mutating state directly (`form.x = 1`)	UI does not update	Always create new objects with spread syntax.
Subscribing to the whole store	Whole app re-renders on every change	Use selectors; `useShallow` for several values.
Hard-coded API URL	Works only on your laptop	Use `VITE_API_URL` and `.env` files.
Forgetting CORS	“Blocked by CORS policy” in the console	Allow the exact origin on the backend.

Pitfall	Symptom	Prevention
Trusting browser validation	Invalid data reaches the API	Keep server-side validation (already in place).
No loading or error state	Frozen or blank screen	Model the request with a `status` value.
Race conditions	An old response overwrites a new one	Abort the previous request and compare request identity.
Secrets in `VITE_` variables	Keys visible to every visitor	Keep secrets on the server.
`key={index}` **in lists**	Wrong row updates or loses state	Use stable ids.
Effects without cleanup	Memory leaks, updates after unmount	Return a cleanup function from `useEffect`.

Final Checklist

- ☐ Project created with Vite, React and TypeScript in strict mode
- ☐ Folders follow the layering rule; no component imports from `api`
- ☐ Types mirror the backend schema; options live in `constants`
- ☐ One API client with timeout, typed responses and readable errors
- ☐ CORS configured with exact origins; environment files in place
- ☐ Zustand store with status, validation, cancellation, persist and devtools
- ☐ Reusable UI components are generic and free of business logic
- ☐ All states designed: idle, loading, success, validation error, request error, empty, offline
- ☐ Keyboard and screen-reader checks passed; contrast verified in both themes
- ☐ Tests pass and the production build succeeds
- ☐ Deployed over HTTPS with matching `ALLOWED_ORIGINS`
- ☐ README explains how to run, test and deploy the project

APPENDIX

Appendix A · Learning Self-Assessment

Tick a skill only when you can do it **without looking at the guide** and can explain it to someone else in one minute.

- ☐ Explain what a component, a prop and state are
- ☐ Explain why `UI = f(state)` and why state must not be mutated
- ☐ Write a TypeScript interface and a union type from an API contract
- ☐ Call an API with `fetch`, handle errors and cancel a request
- ☐ Explain CORS and fix a CORS error
- ☐ Create a Zustand store, a selector and an async action
- ☐ Choose between local state, global state and server state
- ☐ Build a controlled form with validation and accessible errors
- ☐ Render a list with stable keys and conditional views
- ☐ Write a custom hook with an effect and cleanup
- ☐ Use CSS variables, Grid and a media query; add dark mode
- ☐ Write a unit test with a mocked module
- ☐ Read the browser Network tab to debug a request
- ☐ Build and deploy a static site and configure environment variables

Appendix B · Complete Stylesheets

LISTING 31 · src/styles/base.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--bg);
  color: var(--text);
  font:
    16px/1.5 system-ui,
    -apple-system,
    "Segoe UI",
    Roboto,
    sans-serif;
}

h1,

.eyebrow {
  margin: 0;
  font-size: 0.75rem;
  letter-spacing: 0.12em;
  text-transform: uppercase;
```

```
  color: var(--accent);
}
```

LISTING 32 · src/styles/layout.css

```
.topbar {
  background: var(--surface);
  border-bottom: 3px solid var(--accent);
}

.topbar__inner {
  max-width: 1040px;
  margin: 0 auto;
  padding: 18px 20px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}

.topbar h1 {
  font-size: 1.6rem;
  color: var(--text);
}

.page {
  max-width: 1040px;
  margin: 24px auto;
  padding: 0 20px;
  display: grid;
  grid-template-columns: 1.3fr 1fr;
  gap: 20px;
  align-items: start;
}

.stack {
  display: grid;
  gap: 20px;
}

@media (max-width: 800px) {
  .page {
    grid-template-columns: 1fr;
  }
}

.grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 14px 16px;
}

@media (max-width: 520px) {
  .grid {
    grid-template-columns: 1fr;
  }
}

.footer {
  max-width: 1040px;
  margin: 8px auto 32px;
  padding: 0 20px;
  font-size: 0.78rem;
  color: var(--muted);
}
```

LISTING 33 · src/styles/components.css

```
h2 {
  margin: 0;
  font-family: Georgia, "Times New Roman", serif;
}

.badge {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  font-size: 0.8rem;
  padding: 4px 12px;
  border: 1px solid var(--border);
  border-radius: 999px;
  color: var(--muted);
}

.dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: var(--muted);
}

.badge--online .dot {
  background: var(--ok);
}

.badge--offline .dot {
  background: var(--danger);
}

.card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 20px;
}

.card__header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 14px;
}

.card__header h2 {
  font-size: 1.15rem;
}

.field {
  display: grid;
  gap: 4px;
}

.field label {
  font-size: 0.82rem;
  font-weight: 600;
  color: var(--muted);
}

select,

input {
  font: inherit;
  padding: 9px 10px;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: var(--surface);
  color: var(--text);
  width: 100%;
}
```

```
select:focus-visible,

input:focus-visible,

.btn:focus-visible,

.history button:focus-visible {
  outline: 3px solid color-mix(in srgb, var(--accent) 55%, transparent);
  outline-offset: 1px;
}

[aria-invalid="true"] {
  border-color: var(--danger);
}

.field__error {
  margin: 0;
  font-size: 0.8rem;
  color: var(--danger);
}

.actions {
  display: flex;
  gap: 10px;
  margin-top: 20px;
}

.btn {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  font: inherit;
  font-weight: 600;
  padding: 10px 18px;
  border-radius: 8px;
  border: 1px solid transparent;
  cursor: pointer;
}

.btn--primary {
  background: var(--primary);
  color: var(--primary-text);
}

.btn--ghost {
  background: transparent;
  color: var(--text);
  border-color: var(--border);
}

.btn:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.spinner {
  width: 14px;
  height: 14px;
  border-radius: 50%;
  border: 2px solid currentColor;
  border-right-color: transparent;
  animation: spin 0.7s linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

.muted {
  color: var(--muted);
  margin: 4px 0;
```

```
}

.result__value {
  font:
    700 2.4rem/1.1 Georgia,
    serif;
  margin: 4px 0;
  color: var(--primary);
}

.result__meta {
  font-size: 0.78rem;
  color: var(--muted);
  margin: 10px 0 0;
}

.alert {
  background: var(--danger-bg);
  color: var(--danger);
  padding: 10px 12px;
  border-radius: 8px;
  margin: 0;
}

.skeleton {
  height: 64px;
  border-radius: 8px;
  background: linear-gradient(90deg, var(--border), transparent, var(--border));
  background-size: 200% 100%;
  animation: shimmer 1.2s infinite;
}

@keyframes shimmer {
  from {
    background-position: 200% 0;
  }
  to {
    background-position: -200% 0;
  }
}

.history {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
  gap: 8px;
}

.history button {
  width: 100%;
  text-align: left;
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 0 12px;
  padding: 10px 12px;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: transparent;
  color: inherit;
  font: inherit;
  cursor: pointer;
}

.history button:hover {
  border-color: var(--accent);
}

.history__sub {
  grid-column: 1;
  font-size: 0.78rem;
  color: var(--muted);
}
```

```
.history__value {
  grid-column: 2;
  grid-row: 1 / span 2;
  align-self: center;
  font-weight: 700;
}

@media (prefers-reduced-motion: reduce) {
  .spinner,
  .skeleton {
    animation: none;
  }
}
```

Appendix C · References

1. React documentation, “Learn React” and “Thinking in React”: react.dev
2. Zustand documentation, “Getting started” and “TypeScript guide”: zustand.docs.pmnd.rs
3. Vite documentation, “Env variables and modes” and “Building for production”: vite.dev
4. TypeScript Handbook: typescriptlang.org/docs
5. MDN Web Docs: “Fetch API”, “AbortController”, “CORS”, “CSS custom properties”: developer.mozilla.org
6. W3C WAI-ARIA Authoring Practices Guide: w3.org/WAI/ARIA/apg
7. Vitest documentation: vitest.dev · Testing Library: testing-library.com
8. web.dev, “Learn Accessibility” and “Learn Responsive Design”: web.dev/learn